

CSCI 445: An Evaluation of Common Vulnerabilities in Android Applications Using Static Analysis

Christine Hwang, Mia Skibinski

chwang02@wm.edu, mcskibinski@wm.edu

Abstract

Android application security is a growing area of study as mobile applications increasingly manage sensitive assets and inter-app communication. However, Android applications may expand their attack surface through SSL/TLS API misuse, risky combinations of Internet access and privacy-sensitive permissions, over-privileged permission requests, and unprotected exported components. This paper addresses these risks by evaluating 199 Android APKs through four different experiments corresponding to these four risks. Our methodology combines static analysis using Malloroid and Androguard with LLM classification through Google Gemini. It is then followed by manual analysis of a random sample to observe LLM classification reliability. The script experiments find that approximately 33.67% of applications contain likely SSL/TLS misuse, 95% request potentially sensitive Internet and PII-related permission combinations, 98% contain potentially unused permissions, and 66.3% include exported components without protections that may expose application interfaces. Manual analysis was then used to calculate the precision of alerts labeled by Gemini as true positives. The key takeaway is that static analysis provides an effective starting point for identifying Android security risks, but its findings require additional interpretation in some form because automated vulnerability detection frequently flags behavior that may be benign, necessary, or otherwise non-exploitable.

1 Introduction

With Android devices accounting for close to three-quarters of the global marketshare, the development and use of Android apps has been core to its success. Devices with application functionalities range from handheld phones, fitness devices, televisions, and car systems. With the growth of functionalities and capabilities, more attack surfaces for malicious actors have been created. The Google Play Store alone hosts over three million applications. Layers of complexity are added as devices host multiple applications that interact with each other in addi-

tion to apps interacting with the devices themselves. This paper evaluates three different domains of vulnerabilities within Android applications to experiment with, specifically SSL/TLS use, Android Permissions, and Interface Vulnerabilities.

SSL (Secure Sockets Layer) is the predecessor of TLS (Transport Layer Security) which is designed to provide secure communication over a network. By default, network communications are plaintext. This means an actor in between a client and a server can read, record, or modify data in transit without being noticed from either side. TLS was designed to address these issues and guarantee confidentiality, integrity, and authentication. By guaranteeing confidentiality, encryption is used to ensure data is not seen by anyone except for the endpoints of communication. Integrity ensures that the sent data has not been modified in transit. Authentication guarantees that the client is talking to the intended server and not an impersonator. Authentication is achieved with a chain of digital certificates signed by Certificate Authorities (CA) whose root certificates are trusted by the Android system by default. A severe form of SSL/TLS API misuse that will be evaluated in this investigation relates to the custom `X509TrustManager` whose `checkServerTrusted` method contains no validation logic. This is a common practice used for code in development and testing environments where connecting to a local server with a self-signed certificate would inhibit a connection. However, this practice completely eliminates certificate authentication, thus breaking SSL/TLS' guarantee of authentication. Another pattern misuse that is examined in this paper relates to replacing the Android default `HostnameVerifier` with a custom implementation where the `verify` method returns true unconditionally. This means that any hostname for any certificate is accepted.

The goal of analysis pertaining to Android permissions is to find apps that are over-privileged, as systems should only provide the necessary rights to perform functions and nothing more in order to reduce the attack vector of applications. Android's permission system is built on the principle of least privilege, where applications are expected to declare and request only the permissions that are nec-

essary for their intended functionality. Permissions that an application intends on using are declared in the app’s manifest. Android has established a permission system with protection levels. There are several levels; however, this research’s scope will be within normal and dangerous permissions. Normal permissions are those that the platform has judged to be carrying low-risk in isolation. They do not need explicit permission from the user and are automatically granted at install time. Dangerous permissions are those that potentially have the ability to expose sensitive user data or device resources. They are not like normal permissions and are granted at runtime. The system presents a user with dialog of the permission and gives them the choice to allow or deny it. A subset of dangerous permissions are also analyzed in this research’s experimentation. Hard-restricted dangerous permissions have restrictions beyond the runtime requirement. The installer must explicitly allow the permission for that specific package, without regard to user consent. The Internet permission, counterintuitively, is a normal permission, meaning that it is granted automatically and silently to any application that declares it. Combinations of permissions can be a challenge for permission-based security. Based on normal and dangerous permissions, this experiment evaluates how combining them with internet permissions could potentially create an exploitable attack surface. Previous work has been done on finding over-privileged apps, such as UC Berkeley’s “Android permissions demystified,” where 1/3 of applications analyzed with their tool Stowaway did not follow guidelines relating to least privilege [6]. Our paper’s experiment specifically looks at combinations of permissions that relate to Personally Identifying Information as this sensitive information combined with Internet could have negative implications.

Android application components such as activities, services, broadcast receivers, and content providers can be designated as accessible to external applications through `android:exported="true"` in the `AndroidManifest.xml`. Although not all cases of `android:exported="true"` without proper protection guards are dangerous, this exposure to external applications can potentially lead to Interface Vulnerabilities. The potential threats of having an exported attribute include an expansion of the app’s attack surface that could cause login bypass and privilege escalation. A component could be accessible to external callers in the absence of permission grants. Exported content providers without read and write permissions expose their interfaces to provide direct access to potentially sensitive databases, file paths, or backend data sources without caller authentication. Permission protection ameliorates this concern by not allowing a third-party application to have access to functionality.

The remainder of this paper proceeds as follows. Section 2 overviews the structure of our experimentation and analysis. Section 3 describes the design of our experiments. Section 4 evaluates our findings. Section 5 discusses additional questions and components of the experiment. Section 6 describes related work. Section 7 concludes.

2 Overview

These five research questions were evaluated in four experiments. Questions 1 and 2 were combined into Experiment 1, while Experiment 2 was correlated with Question 3, Experiment 3 was correlated with Question 4, and Experiment 4 was correlated with Question 5.

1. Do the apps allow all Hostnames by overriding the Hostname Verifier?
2. Do the apps override default trust certificates using custom Trust Managers?
3. Do the apps use the potentially harmful combination of Internet and personal information permissions?
4. Do the apps use all the permissions they request?
5. Are Activities set to `android:exported="true"` without being protected by a specific permission?

The general structure of each experiment holds to certain steps as in Figure 1. First, APK files were parsed using Androguard which allowed for access to the bytecode for static analysis. Scripts were then specifically made for each research question and run on each application to flag potential vulnerabilities. The output from the static script analysis, if it did not hold to the research question’s predefined ruleset, was then input into a large language model (in our case, Google Gemini) that flagged these vulnerabilities as false or true positives. These results were then randomly sampled with manual review to validate accuracy.

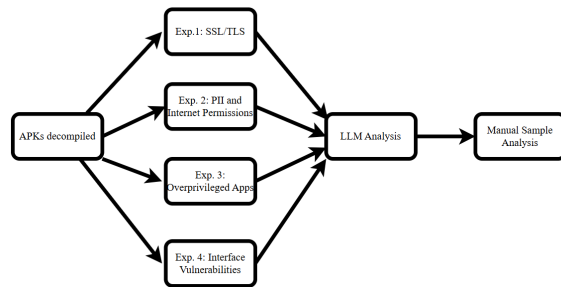


Figure 1: Flow chart of experimental approach

Androguard is an open source Python library that operates on APK files to parse the native DEX format. Androguard allows for writing python scripts to traverse through the accessible full class hierarchy, method signatures, API call graphs, control flow, and manifest structure [3].

Malldroid is a static analysis tool for detecting SSL/TLS API misuse in Android applications. It analyzes Android application bytecode to search for patterns of misuse relating to custom `TrustManager` and custom `HostnameVerifier` implementations. Malldroid complements Androguard, as it is implemented on top of its infrastructure [5].

199 Android applications were provided to the Mobile Application Security Class by Professor Nadkarni. The dataset includes a wide range of applications with different purposes which have been taken into account for determining if some features, although seeming like a vulnerability, are necessary for the application’s core functionality.

3 Design

3.1 Experiment 1: SSL/TLS API Misuse

The first experiment focuses on SSL/TLS vulnerabilities. Specifically, it scans the 199 apps for custom Trust Managers and custom Hostname Verifiers, two common instances of SSL/TLS misuse. We used a previously-written script that utilizes Malldroid, which is built on Androguard, to reverse engineer the APKs [7]. The original script scans for five different SSL/TLS errors, and we narrowed the scope down to only custom Trust Managers and custom Hostname Verifiers. However, one of the biggest challenges in SSL/TLS error detection is avoiding false positives. An output of a script such as the one we adapted from the Malldroid script can have an unmanageable amount of alerts. To mitigate this issue, we added functionality to scan for related runtime usage of the suspicious classes flagged by the original script. By doing so, we are able to categorize our alerts as HIGH risk probable vulnerabilities and LOW risk potential false positives.

We then ran our script on the APKs in five separate batches, four of them containing 40 APKs and the fifth one containing 39 APKs, in order to split up the long runtime of the script. As the script analyzes each APK, it outputs a plaintext file detailing each error found within the app. Specifically, it displays the APK’s name, package identifier, and a summary of the findings as the header. For each finding, it categorizes it as either HIGH risk or LOW risk, depending on its runtime evidence. Findings are classified as HIGH risk when runtime evidence shows that the suspicious component appears to be connected to HTTPS configuration code such as `SSLContext.init(...)` or

`setSSLSocketFactory(...)`. This indicates the custom Trust Manager or Hostname Verifier is likely used during real network communication. In contrast, findings are classified as LOW risk when the script identifies suspicious SSL/TLS code but does not find any evidence that it is used in runtime HTTPS setup. Additionally, each finding contains the type of alert, class and method location, runtime evidence, other references to the class, and the specific Java source code. The script creates a `reports` directory and outputs a plaintext file for each individual app as well as a batch summary. The batch summary details which APKs were successfully analyzed, skipped for no Internet permission, or failed to analyze as well as a count of apps with HIGH-risk custom trust managers and a count of apps with HIGH-risk hostname verifiers.

Once the script finished running on a batch of apps, we then analyzed each plaintext file using Google’s Gemini while running the script on the next batch. For each alert in the plaintext output file, we prompted Gemini with “Does this alert seem to be a false positive, meaning the vulnerability is not actually exploitable or if it is exploited it does not cause harm?” and pasted the alert information. Each response from Gemini was then recorded as a True or False positive, and each False positive was categorized as being necessary for the app’s functionality, apart of dead/test code, or apart of secure logic/libraries. An app was only labeled as vulnerable if a True positive was found. This process was then repeated on all remaining apps.

The final step in our process was to evaluate the accuracy of Gemini in its labeling. To do this, we took a random sample of 20 apps to represent our dataset. We then manually analyzed every alert that was flagged within each app. Using JADX to decompile the APKs, we physically looked through the source code to verify whether or not each alert was a True or False positive based on our previous conditions. Once the manual analysis by us was finished, the total statistics were gathered, including the script output, the results from Gemini, and the results from the manual analysis.

3.2 Experiment 2

Experiment 2 was designed to specifically look for potential vulnerabilities relating to permissions with personally identifying information (PII) and Internet permission. Using Android documentation, permissions were categorized into normal and dangerous when combined with Internet.

Since permissions for functionalities that are intended to be used are declared by developers in the Android manifest, this script specifically traversed through each app’s `AndroidManifest.xml` to find combinations between PII permissions and Internet. The script was designed to have

Table 1: Representative Android permissions associated with PII risk when combined with INTERNET.

Permission	Category	PII Risk
<i>Normal permissions</i>		
FOREGROUND_SERVICE_	Persistent access	Biometric
CAMERA		
FOREGROUND_SERVICE_	Persistent access	Biometric
MICROPHONE		
PACKAGE_USAGE_STATS	Behavioral	Usage
<i>Dangerous permissions</i>		
RECORD_AUDIO	Biometric	Biometric
ACCESS_FINE_LOCATION	Location	Location
READ_SMS*	Communications	Communications
READ_CONTACTS	Identity	Identity
READ_MEDIA_IMAGES	Stored media	Personal media
BLUETOOTH_SCAN	Proximity	Device association

* Hard-restricted permission. All permissions are prefixed with `android.permission.`. The full list contains n permissions spanning biometric, location, communications, identity, stored media, and proximity categories.

an output that showed the number of normal permissions and dangerous permissions that were combined with Internet permissions. The script also outputs a risk level, which could be high (there is a dangerous permission present in addition to a hard restriction), medium (dangerous permissions are present), low (normal permissions are combined with internet), and none (if internet is absent or no categorized permissions are found). The output after the end of running all of the scripts presents how many of the 199 applications have at least one potential permissions vulnerability and total number of instances from the entire test dataset based on these guidelines.

After running this static analysis script on all of the applications, Google Gemini was prompted specifically with, "Based on this given static analysis data, determine which vulnerabilities here are true or false positive based on app functionality, context, and purpose." Each application's alerts were separately prompted into Gemini for analysis. After running each result from the scripts with the LLM, the updated number of false and true positives were logged.

3.3 Experiment 3

Experiment 3 pertains to over-privilege in applications. Applications being over-privileged was determined with permission mapping. This allowed us to see if the apps use the permissions that they request in their manifest. This experiment was structured by creating a script that implemented a dictionary. The key values were permissions that could be requested by apps and the values were specific strings that would be necessary to prove that that permission was used in the application. If the value(s) were missing, then the application would be flagged as having an unused permission and be categorized as over-privileged.

Similar to experiment 2, the script needed to look at

Table 2: Sample permission-to-API pattern mapping

Permission	API Patterns
<i>Normal permissions</i>	
FOREGROUND_SERVICE_	startForeground,
CAMERA	FOREGROUND_SERVICE_TYPE_CAMERA
PACKAGE_USAGE_STATS	UsageStatsManager,
	queryUsageStats
<i>Dangerous permissions</i>	
CAMERA	android/hardware/camera2,
	CameraManager
READ_CONTACTS	ContactsContract,
	ContentResolver
READ_SMS	android/provider/Telephony,
	content://sms

All permissions are prefixed with `android.permission.`. The full mapping contains n permissions across normal and dangerous categories.

the Android Manifest to find what permissions were requested. However, this experiment had to go a step further and look through the rest of the application to ensure the requested permission was being used. The produced output was a list of the application's requested permissions that appear to be unused. Additionally, permissions that should be further examined with manual review were flagged. Like experiment 2, after all the applications were run with static scripting, the total number of apps found with these patterns were aggregated.

This output was then input to Google Gemini for analysis for false and true positives. Each application's alerts were separately prompted into Gemini for analysis. The specific prompt was the same as experiments 2 and 4 with, "Based on this given static analysis data, determine which vulnerabilities here are true or false positives based on application functionality, context, and purpose." After running each of the static outputs with the LLM, this new data was logged.

3.4 Experiment 4

Experiment 4 relates to interface vulnerabilities, specifically when `android:exported="true"` without meaningful protections. The script for this experiment was designed keeping in mind that there are instances where it is necessary for `android:exported="true"` without permissions. For example, the main activity for launching an application is a necessary function that does not need additional protection.

The output from this experiment was analyzed through a large language model to flag false and true positives, using the same prompt that was used in experiments 2 and 3. Each application's alerts were separately prompted into Gemini for analysis. This data was then manually analyzed to find if the script acted correctly within its scope and correctly identified or did not identify exports as possible vulnerabilities. Manual analysis was also used to

determine if the export was necessary for functionality.

4 Evaluation

4.1 Experiment 1 Results

Table 3 displays the overall final results of Experiment 1. After running the script on all 199 APKs, the output displayed a total of 851 alerts. Accordingly, 165 apps had some form of alert, while 34 apps did not appear to have any SSL/TLS API misuse, 5 of them due to no Internet access permission. Within the 165 flagged apps, 60 of them had at least one HIGH-risk custom Hostname Verifier and 37 of them had at least one HIGH-risk custom Trust Manager. As we said before, an alert is labeled HIGH risk if the script can find runtime evidence for the suspicious code. Overall, the script identified 67 apps as vulnerable, defined as apps containing at least one HIGH-risk alert, and it identified 132 of the apps as not vulnerable, defined as apps containing no HIGH-risk alerts or no alerts at all. Based on the script output, 33.67% of the apps were found to be vulnerable. This indicates that approximately one-third of the dataset contains typical patterns associated with SSL/TLS API misuse.

Table 3: RQ1 and 2: SSL API Misuse

Category	Script Output	Gemini Analysis
Custom Hostname Verifier	60	56
Custom Trust Manager	37	55
Not vulnerable	132	130
Vulnerable	67	69

All data entries are a number of apps

The next step of our procedure following initial static analysis is analysis using an LLM. For this particular study, we chose Gemini to analyze our results. After having Gemini analyze each individual alert for all 199 apps, it found that a total of 185 alerts were True positives and 666 alerts were False positives. From this analysis, 69 apps were found to have at least one True positive alert, indicating that the app is vulnerable. Within these 69 apps, 56 instances of True positive custom Hostname Verifiers were found, and 55 instances of True positive custom Trust Managers were found. Conversely, 130 apps had all False positive alerts, indicating that the app is not vulnerable. Overall, the Gemini output found that 34.67% of the apps were vulnerable, a difference of only 1 percentage point in comparison to the script output. This suggests that the script is accurate in labeling apps as vulnerable. The small increase in the Gemini analysis reflects cases where the script identified some alerts as False positives that were valid True positives. Further comparison between the script output and the Gemini analysis shows a negligible difference between the number of apps

found with at least one custom Hostname Verifier. The larger discrepancy between the number of apps with at least one custom Trust Manager indicates that many of the custom Trust Manager alerts flagged by the script as LOW risk were actually True positives. However, the negligible difference of apps labeled as vulnerable indicates that these True positives were present in apps already containing another True positive alert. Therefore, only two additional apps were flagged as vulnerable in comparison to the script output.

The final step of our procedure uses by-hand manual analysis to verify a random sample of Gemini-labeled alerts. After taking a random sample of 20 apps, just over 10% of our original dataset, we manually analyzed a total of 65 alerts, both HIGH and LOW risk, in the 20 apps. We found that 56 of the alerts from the script were labeled the same way by both the LLM and our manual analysis while 9 alerts were labeled differently. This indicates that 86.15% of the alerts labeled by Gemini were supported according to our manual analysis, while 13.8% of them were labeled incorrectly. Interestingly, all of the mislabeled alerts were labeled as True positives by Gemini but were actually False positives according to our manual analysis. Assuming these 20 apps are a representative sample of the original dataset, approximately 733 out of the 851 total alerts flagged by the script were labeled correctly. In contrast, approximately 118 of them were labeled incorrectly. Additionally, our manual analysis found that two of the apps labeled as vulnerable by Gemini analysis were actually not vulnerable, meaning the True positives present in the app were actually False positives. This suggests that full manual analysis of the alerts would lower the overall amount of vulnerable apps, which supports the general heuristic that static analysis outputs a large number of false positives. Overall, Experiment 1 indicates that there are instances where the apps allow all Hostnames by overriding the Hostname Verifier or override default trust certificates using custom Trust Managers, but the prevalence of these issues within our dataset is not significant enough to establish widespread SSL/TLS vulnerabilities of these types.

4.2 Experiment 2 Results

95% of the applications were found to have a potential vulnerability relating to dangerous permission combinations. Experiment 2 yielded 188 flagged vulnerabilities across all 199 application apks.

Using Gemini as the LLM for further analysis, of these vulnerabilities, only two of them were found to be false positives, resulting in a precision percentage of 99.5%.

Manual Analysis of this specific experiment was done on 20 of the 199 apps, amounting to about 10% of the total apps that were in the dataset. The 20 applications

were randomly selected. After looking through each output, it was determined that the script accurately found potentially dangerous combinations as it correctly identified the permissions that were asked to be found in the script in addition to the Internet permission. Any of the combinations present an attack surface, so manual and LLM analysis matched in numbers of true positives.

The high rate of true positives from this analysis is not the baseline for concluding that these applications are vulnerable. Of these 199 applications, many needed permissions that combine Internet and an Android-categorized dangerous permission to execute core functionality. For example, of the 199 applications, about 40% related to hardware control (remote control functions for televisions, etc.) and need functionalities relating to location and WiFi. We cannot determine that an application is completely free of vulnerabilities, but can conclude that there is an increased attack surface with the use of these combinations.

Table 4: Internet + PII Permission Vulnerabilities

Category	Total Flagged	TP	FP	Prec.
Internet + PII	188	187	2	99.5%

4.3 Experiment 3 Results

98.0% of the applications were found to have a potential vulnerability pertaining to unused permissions. Experiment 3 identified 1,018 flagged potential vulnerabilities.

Using Gemini as the Large Language Model for analysis, Gemini flagged 157 of the flagged vulnerabilities in applications as true positives and 861 as false positives, yielding a precision rate of 15.4%.

Manual analysis confirmed that the script successfully found what it aimed to find. However, this is not an indicator of success in finding true positives. Out of 20 applications randomly selected for analysis, 22.4% of the flags were true positives, compared to LLM analysis with a precision of 15.4%. This gap between LLM precision and manual analysis could be an indicator of the LLM being more conservative with categorizing what was a potential vulnerability relating to unused permissions. Upon further analysis of the types of applications and their necessary functionalities, it can be inferred that the script was not comprehensive to all of the possible attack surfaces.

The implications of this data having a high rate of false positives could also include not factoring in significant components when creating the static scripts. As seen with this small precision rate, it can be inferred that the script lacked exceptions for permissions commonly satisfied by third party libraries rather than application level code. One example of a component that may not have been ac-

counted for is a third party feature. Third party features such as OkHTTP or Firebase use permissions that are requested in the manifest. However, the script mapping of permissions to API calls may not have been comprehensive enough. A more robust implementation would have maintained an exclusion list of permissions routinely satisfied by common SDKs.

Table 5: Unused Permission Vulnerabilities

Category	Total Flagged	TP	FP	Prec.
Unused perms.	1,018	157	861	15.4%

4.4 Experiment 4 Results

66.3% of the total applications were flagged with having a potential vulnerability relating to unguarded exports. Experiment 4 with static scripting flagged 1,100 vulnerabilities with exported components across all of the analyzed applications.

After running the static script output in Google Gemini, the new data revealed that 906 of the 1,100 flagged vulnerabilities were categorized as true positives and 178 were categorized as false positives. This yielded a precision rate of 82.4%.

Manual analysis confirmed that the script correctly flagged `android:exported="true"` appropriately, although there appeared to be minor discrepancies between LLM analysis and manual analysis. Manual analysis on a sample size of 20 applications confirmed 92% script precision, slightly above the LLM’s 82.4%, suggesting that the LLM was modestly conservative on flagging false positives. The LLM’s primary value-add over the scripting was recognizing surrounding code to be non-exploitable by having context of the functionality of the application rather than immediately categorizing an instance as a vulnerability because an export was unguarded.

Table 6: Interface Vulnerabilities

Category	Total Flagged	TP	FP	Prec.
Exp. components	1,100	906	178	82.4%

5 Discussion

There are several factors that should be accounted for when recreating or expanding on this experiment.

First, the use of Large Language Models, although helpful and efficient for large data sets, are not an oracle for determining whether every potential vulnerability is a true or false positive. It was only used as a tool for this

experiment to explore streamlining the process of evaluating a large dataset. There are many variables that need to be accounted for, including, the specific LLM used, the uniformity of answers based on tokens/model, possibilities of hallucination and bias. These factors were not able to be comprehensively reviewed under the constraints of this research experiment. Future experiments could have immensely different results depending on the capabilities of AI models and if the version of AI models is advanced or standard. Model consensus may also relieve these concerns as results can be triaged to find agreed upon results.

Second, these experiments were not comprehensive of every single scenario and functionality an application may have. Absence of evidence does not completely eliminate the possibility that there is a threat. For example, there will be exceptions and use cases where an app’s code looks as though it is a vulnerability but it is absolutely necessary for the application’s functionality. This is a necessary trade off of risk for function.

Static analysis was used in these experiments, but they do not give a complete picture of all the possible threats that an application may have. Dynamic analysis is also an imperative component to finding run time vulnerabilities and possible actions that a user may commit to create vulnerabilities. Thus, the results in some of these experiments cannot be fully declared not vulnerable as they may be vulnerable when running.

Future experiments that branch off of this work and harden this work should include having less variables and striving for a more constant test environment. Experiment 3 would also benefit from being changed to gain a higher rate of precision, most prominently by taking into account third party SDKs and other outside entities that utilize permissions.

These experiments and their results are a culmination of standard techniques as well as assumptions. With this in mind, there is sizeable room for error with data involving scripting analysis, LLM analysis, and manual analysis. However, these numbers more importantly present a larger picture of vulnerabilities and threats in Android application environments.

6 Related Work

Prior work on Android security has shown that mobile applications create many risks simply through ordinary development choices that expand an app’s access to sensitive data, network communication, and external interfaces. Because of the specific structures of these Android applications, security analysis is difficult to automate. Tools must be able to properly reason about the way Android apps run, store information, and communicate with other components or applications. Reaves et al.

describe these challenges as central to Android application analysis. They show that tools must account for life-cycle behavior and inter-app communication in order to analyze them accurately [9].

This project builds on the Reaves et al. line of research by looking for common risky patterns across many applications. Previous systems such as TaintDroid demonstrate why these risks matter. By labeling sensitive data and monitoring its flow through dynamic taint tracking, it found that many applications transmitted sensitive information to outside servers. These findings motivate our focus on combinations of Internet access and PII-permissions [4]. However, our work differs from TaintDroid because of our use of static analysis instead of dynamic analysis.

Static taint-analysis tools such as FlowDroid also motivate our approach. FlowDroid shows that static analysis can be made more precise for Android by analyzing more aspects of the application rather than the source code alone. This supports our use of APK-level analysis by indicating many Android security properties can be studied without executing the app. However, FlowDroid’s own limitations also show why static findings require careful interpretation. The authors explain that instances such as reflection or unknown callbacks can cause missed behavior [1]. Therefore, our work uses static analysis only as a starting point rather than definite exploitability proof.

Research on the “soundness” of Android security tools also shows why these results need careful interpretation. In their paper coining the concept of “soundness”, Livshits et al. explains that a “soundy” analysis tries to be as reliable as possible without sacrificing an excessive amount of accuracy or scalability [8]. Further study of static analysis tools specifically finds that they often trade soundness for precision and performance, and that these tradeoffs are not always fully documented. Bonett et al. created a framework that found hidden flaws in Android static analysis tools and showed that these flaws can carry over into newer tools built off of them [2]. This again motivates one of our main choices in our methods to not treat automated output from our scripts as a final judgment of app vulnerability. Instead, we use additional interpretation, both through LLM and manual analysis, to distinguish between the True and False positives in the output.

Overall, these studies indicate both the importance and the difficulty of Android application security analysis. Dynamic systems such as TaintDroid give strong indications of actual sensitive-data leaks but depend on runtime evidence. On the other hand, static systems such as FlowDroid scale better to APK collections but commonly produce incomplete or overly-broad results. Our project combines parts of both approaches. We use static analysis to study our research questions across many APKs, then use additional classification and review to establish runtime

evidence and exploitability. Our project builds on prior Android analysis research, but it has a different focus and scope. We evaluate how existing static techniques can be combined with LLM and manual review to characterize common security risks in real Android applications.

7 Conclusion

As mobile applications become an important aspect of everyday life, properly securing these systems that may handle sensitive assets becomes an increasingly complex issue. In this paper, we evaluated 199 APKs for SSL/TLS API misuse, risky combinations of Internet access and privacy-sensitive permissions, over-privileged permission requests, and unprotected exported components. The central problem we found across all four experiments was that Android security risks were often more nuanced than our static analysis could reveal. This indicates that vulnerability detection should be a process of interpretation rather than simple pattern matching, because the security significance of a finding depends heavily on context, implementation, and intended functionality. Overall, we expected our procedure to work because static analysis provides surface-level alerts from the large numbers of applications, while LLM classification and manual analysis helps distinguish meaningful risks from benign behavior. The findings from these experiments reinforce that many Android applications contain potential attack surfaces, but they also demonstrated that total automation of the analysis process does not yield accurate results. Therefore, effective mobile security analysis requires a balance between automation and human reasoning. Automation is essential for scale, but interpretation is essential for accuracy. Future work should build on this conclusion by utilizing dynamic analysis, comparing multiple LLM results, and expanding manual validation samples to better determine the scope and limitations of automated reasoning in Android app security analysis.

References

- [1] S. Arzt, S. Rasthofer, C. Fritz, E. Bodden, A. Bartel, J. Klein, Y. Le Traon, D. Oteau, and P. McDaniel. Flowdroid: precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for android apps. *SIGPLAN Not.*, 49(6):259–269, June 2014.
- [2] R. Bonett, K. Kafle, K. Moran, A. Nadkarni, and D. Poshvanyk. Discovering flaws in security-focused static analysis tools for android using systematic mutation. In *Proceedings of the 27th USENIX Conference on Security Symposium*, SEC’18, page 1263–1280, USA, 2018. USENIX Association.
- [3] A. Desnos. Androguard. <https://github.com/androguard/androguard#>, 2012.
- [4] W. Enck, P. Gilbert, S. Han, V. Tendulkar, B.-G. Chun, L. P. Cox, J. Jung, P. McDaniel, and A. N. Sheth. Taintdroid: An information-flow tracking system for realtime privacy monitoring on smartphones. *ACM Trans. Comput. Syst.*, 32(2), June 2014.
- [5] S. Fahl, M. Harbach, T. Munders, L. Baumgärtner, B. Freisleben, and M. Smith. Why eve and mallory love android: an analysis of android ssl (in)security. In *Proceedings of the 2012 ACM Conference on Computer and Communications Security*, CCS ’12, page 50–61, New York, NY, USA, 2012. Association for Computing Machinery.
- [6] A. P. Felt, E. Chin, S. Hanna, D. Song, and D. Wagner. Android permissions demystified. In *Proceedings of the 18th ACM Conference on Computer and Communications Security*, CCS ’11, page 627–638, New York, NY, USA, 2011. Association for Computing Machinery.
- [7] N. Janis. Droid analysis. <https://github.com/NDJSec/DroidAnalysis>, 2024.
- [8] B. Livshits, D. Vardoulakis, M. Sridharan, Y. Smaragdakis, O. Lhoták, J. Amaral, B.-Y. Chang, S. Guyer, U. Khedker, and A. Møller. In defense of soundness: A manifesto. *Communications of the ACM*, 58:44–46, 01 2015.
- [9] B. Reaves, J. Bowers, S. A. Gorski III, O. Anise, R. Bobhate, R. Cho, H. Das, S. Hussain, H. Karachiwala, N. Scaife, B. Wright, K. Butler, W. Enck, and P. Traynor. *droid: Assessment and evaluation of android application analysis tools. *ACM Comput. Surv.*, 49(3), Oct. 2016.